

Performance do código

Na maratona, seu código deve retornar a saída correta para ser aceito. Isso por si só, já é um desafio. No entanto, boa parte do esforço nessas competições é escrever código que seja rápido o suficiente para passar no limite de tempo do juiz.

Os juízes irão executar seu código com uma série de casos de teste onde cada teste deve executar dentro de um limite de tempo (geralmente 2, 1 ou 0,5 segundos). Se o seu código demorar mais do que o limite, ele receberá o veredito **Time Limit Exceeded (TLE)**.

Devemos otimizar nosso código para que ele seja rápido o suficiente.

Exemplo: Soma de 1 até N

Considere o problema de somar todos os números inteiros de 1 até N . Para isso podemos fazer um for percorrendo todos esses números e somando-os dentro de uma variável:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    // como o resultado da soma pode ser grande, usamos um long long.
    long long sum = 0;
    for (int i = 1; i <= n; i++) {
        sum += i;
    }
    cout << sum << endl;
    return 0;
}
```

Para realizar essa soma, o computador irá executar N repetições do laço **for**. Se N for grande, o tempo de execução será grande. Por exemplo, se $N = 10^9$, o código acima demoraria muito para terminar e receberia um TLE.

Ao invés de percorrer os números, podemos utilizar a fórmula clássica da soma de uma progressão aritmética, que resolve o problema em uma quantidade menor de passos:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    // como o resultado da soma pode ser grande, usamos um long long.
    long long sum = 0;
    // fórmula da soma de uma progressão aritmética
```

```
sum = (long long) n * (n + 1) / 2;  
cout << sum << endl;  
return 0;  
}
```

Note que o código acima resolve o mesmo problema e não há laços. Há apenas algumas poucas operações de soma, multiplicação e divisão. O tempo de execução desse código é muito menor e não se importa com o tamanho de N . Ele será aceito mesmo para $N = 10^9$.

Para entender melhor o desempenho de um código, é preciso usar uma forma de medição. A mais comum é a **complexidade** do código, onde fazemos uma estimativa de quantidade de passos que o programa executa e depois simplificamos essa conta. Veremos complexidade nas próximas aulas.

Tente resolver os problemas seguindo as dicas.